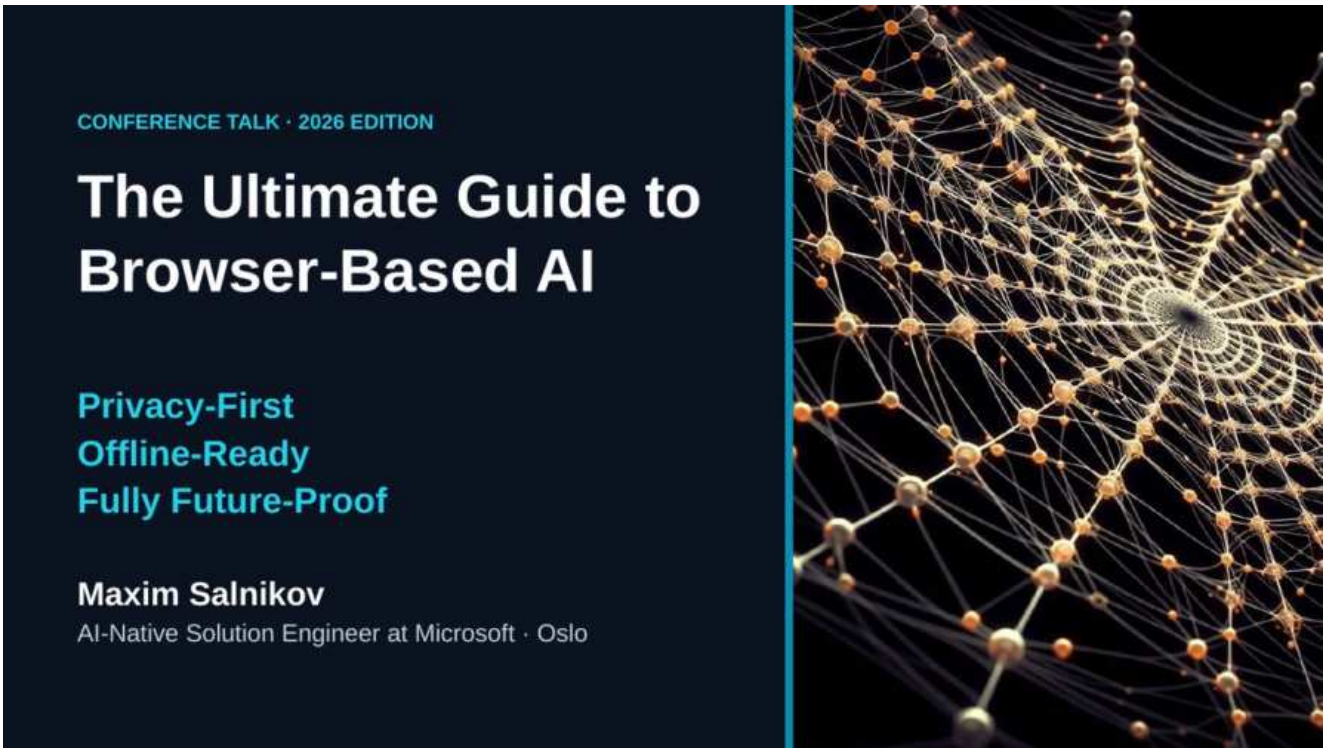


Slide 1 · The Ultimate Guide to Browser-Based AI (title)

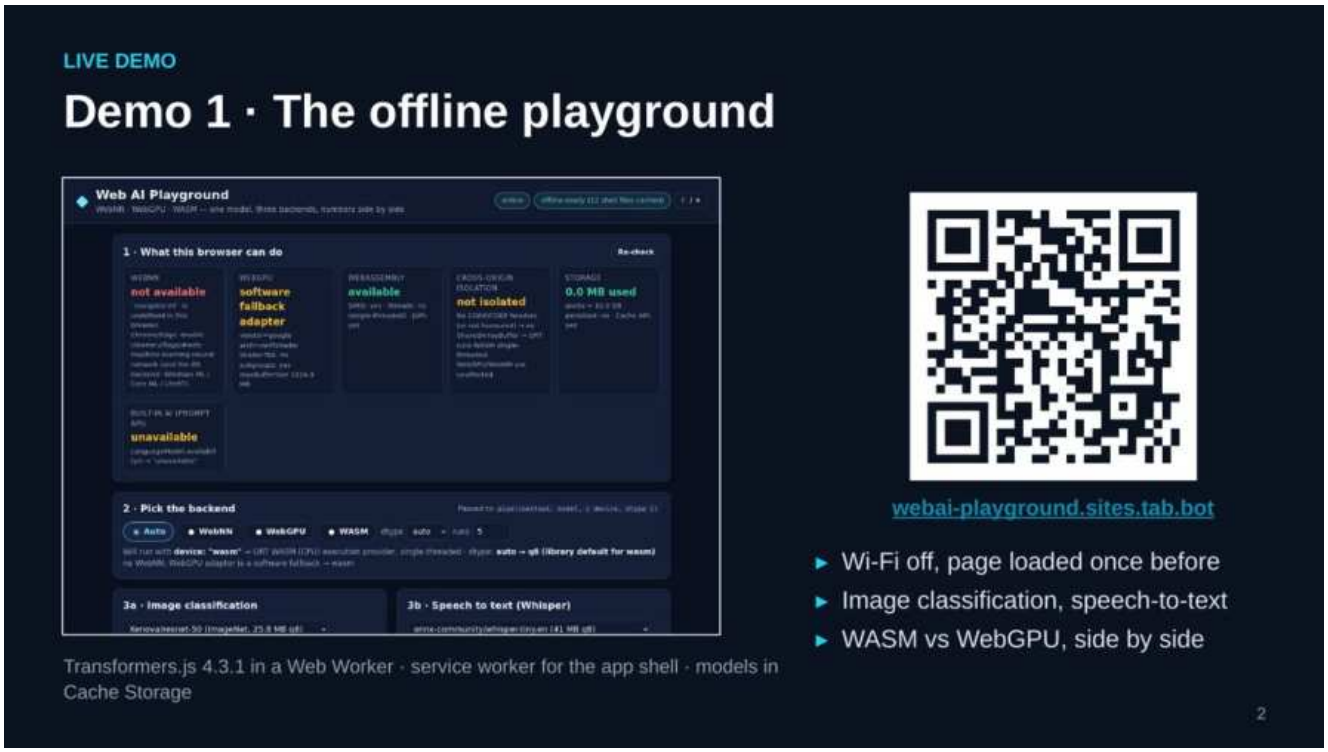
Time budget: 0.5 min (running total 0.5 min)

[Slide up while people sit down. Laptop already has the playground open in a background tab; Wi-Fi still on.]

Hi everyone, I'm Maxim. This talk is about running AI where your users already are: inside the browser, on their own device.

I won't start with slides. I'll start with a demo, because everything I'm going to claim today you should be able to see working.

[Click to the next slide.]



Slide 2 · Demo 1 · The offline playground

Time budget: 3 min (running total 3.5 min)

[Switch to the browser. BEFORE THE TALK: the page was loaded once with all models run, so they sit in Cache Storage. Now turn Wi-Fi OFF on stage, visibly.]
[Verified 16 Oct on the published URL: after one online run, with the network off and the page reloaded, ResNet-50 loaded from Cache Storage and classified offline. Still do a quick pre-stage check, as with any live demo.]

This is the Web AI Playground. I rebuilt it this month on Transformers.js 4.3.1.

Look at the top panel first. The page asks the browser what it has: WebNN, WebGPU, WebAssembly threads and SIMD, storage. No guessing.

[Pick the corgi sample. Device: WASM. Click Classify.]

Pembroke Welsh corgi. That ran on the CPU through WebAssembly. Load time, first run, average of five runs, all on screen.

[Switch device to WebGPU, classify again.]

Same model, same page, now on the GPU. And every result goes into this table, so we can compare backends side by side.

[Scroll to the speech card, run the JFK clip.]

And this is Whisper tiny, speech to text, in the same tab. "Ask not what your country can do for you..."

No server touched any of this. Everything you just saw is the standard web platform plus one import. The QR code is on screen if you want to follow along on your laptop.

[Wi-Fi back on. Back to slides.]



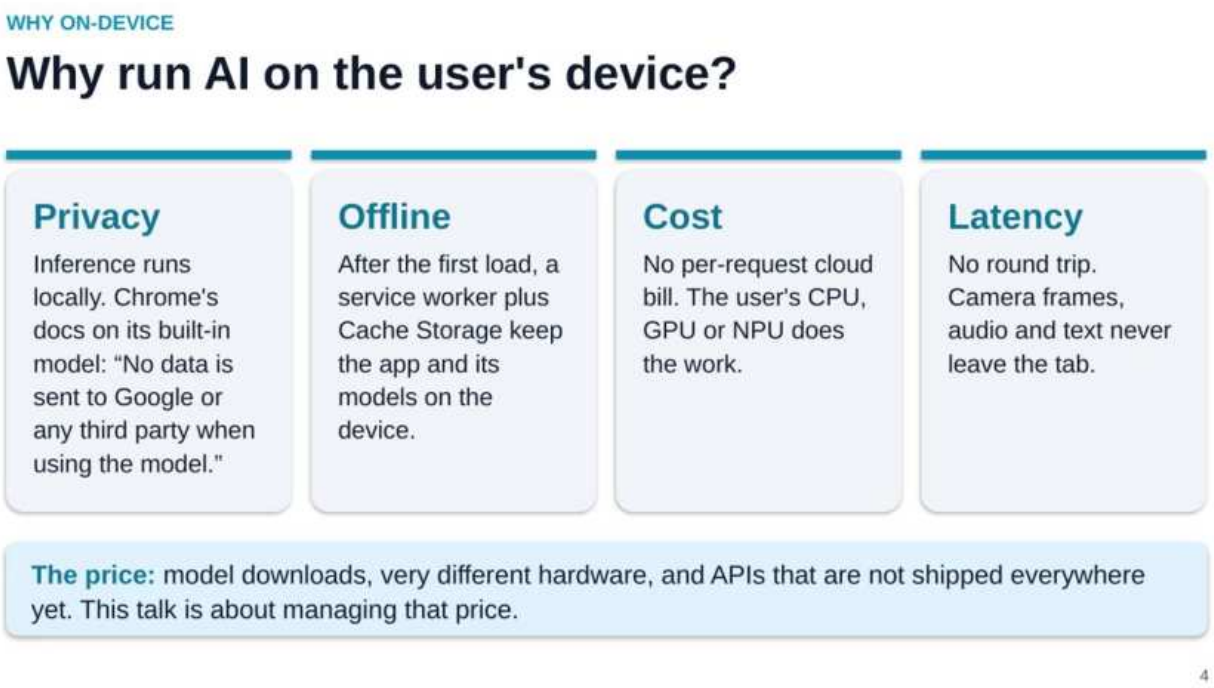
Slide 3 · Hi, I'm Maxim

Time budget: 0.5 min (running total 4 min)

Quickly about me. I'm an AI-Native Solution Engineer at Microsoft, based in Oslo. I've been building on the web since the 90s.

The part that matters for today: I'm a member of the W3C Web Machine Learning Community Group. That's where WebNN, the Prompt API and WebMCP get incubated. So I'll tell you where things really stand, including the parts that are not shipped yet.

If you like this topic, my newsletter is at isainative.substack.com. And I wrote a book on WebMCP, "Tools, Not Clicks". We'll get to WebMCP at the end.



Slide 4 · Why run AI on the user's device?

Time budget: 1.5 min (running total 5.5 min)

Why bother? The abstract names three trade-offs of cloud AI, and I add a fourth.

Privacy. The data stays on the device. Chrome's documentation for its built-in model says it plainly: no data is sent to Google or any third party when you use the model.

Offline. Once the app and the model are cached, you don't need the network. A train, a plane, a hospital basement.

Cost. You don't pay per token. The user's hardware does the work.

Latency. No round trip. That matters most for live media: camera frames, microphone audio.

But I won't pretend it's free. You pay with model downloads, with hardware that varies wildly between users, and with APIs that are not shipped in every browser yet. The rest of this talk is about paying that price smartly.



Slide 5 · What changed: 2023 → 2026

Time budget: 1.5 min (running total 7 min)

What changed since the first version of this talk? Three things.

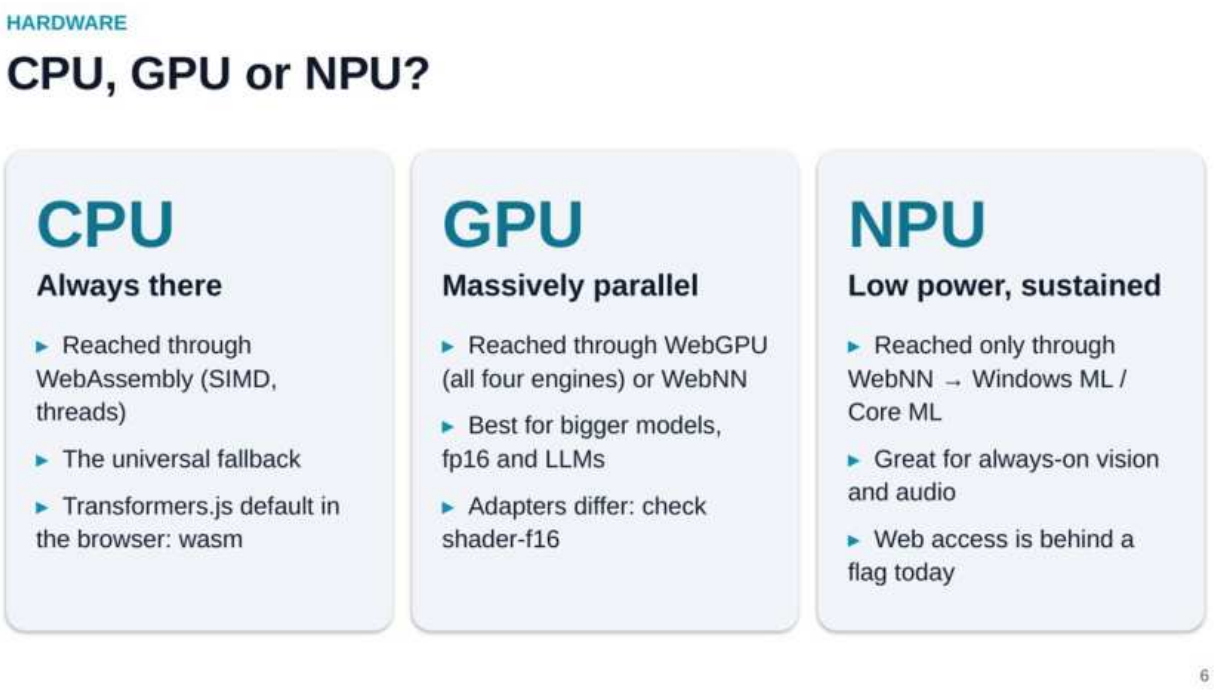
2023: WebGPU shipped in Chrome and Edge 113. For the first time a web page could use the GPU for general compute, not just graphics.

2024: WebNN went into developer trial in Chrome 125, behind a flag. That's the API that can reach the NPU.

2025: browsers started shipping their own models. Translator, Language Detector and Summarizer went stable in Chrome 138. And WebGPU reached Firefox 141 and Safari 26, so it's in all four engines now.

2026: the Prompt API, a general language model API, is stable on the web in Chrome 148. WebNN reached Candidate Recommendation, with the snapshot from January.

So: GPUs became reachable, NPUs showed up in laptops, and browsers now bring their own models.



Slide 6 · CPU, GPU or NPU?

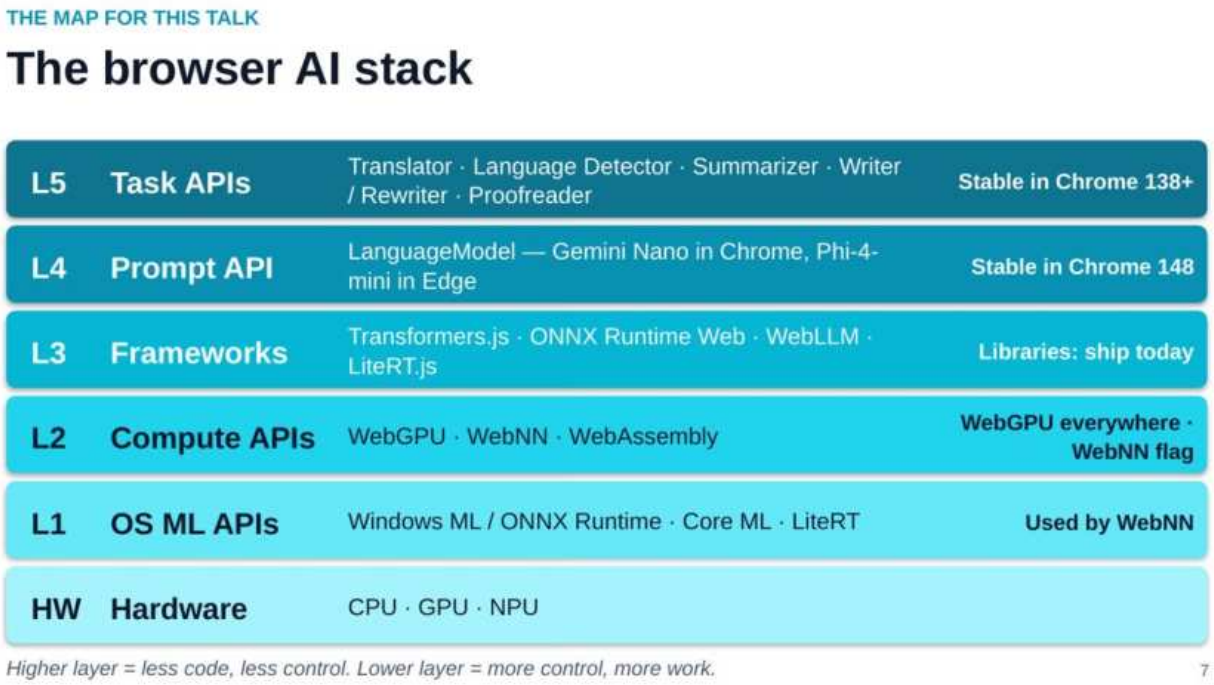
Time budget: 1.5 min (running total 8.5 min)

I kept this slide from the original version because the question is still the same: which chip runs your model?

The CPU is always there. On the web you reach it through WebAssembly, with SIMD and threads. It's the universal fallback, and it's what Transformers.js uses by default in the browser.

The GPU is massively parallel. WebGPU is now in all four engines, and WebNN can target the GPU too. That's where bigger models and LLMs live. But GPUs differ: always check whether the adapter supports shader-f16 before you pick fp16 weights.

The NPU is the newcomer. Low power, built for sustained inference: background blur, live captions, always-on vision. On the web, the only door to the NPU is WebNN, which hands the work to Windows ML or Core ML. And that door is behind a flag today. I'll show you exactly where it stands.



Slide 7 · The browser AI stack

Time budget: 1.5 min (running total 10 min)

This is the map for the rest of the talk. Six layers, top to bottom.

At the top, L5: task APIs built into the browser. Translate this, summarize that. One line of code, the browser brings the model.

L4: the Prompt API. A general language model in the browser: Gemini Nano in Chrome, Phi in Edge.

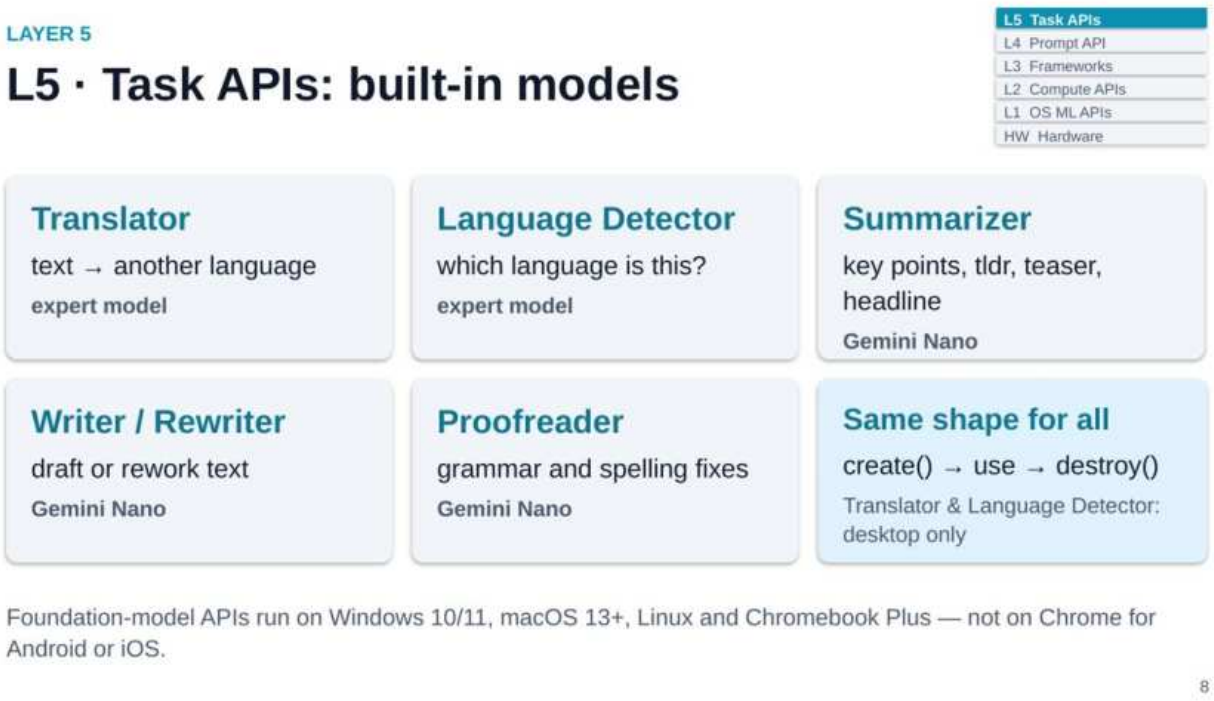
L3: frameworks you ship yourself. Transformers.js, ONNX Runtime Web, WebLLM, LiteRT.js. You choose the model.

L2: the compute APIs those frameworks sit on. WebGPU, WebNN and WebAssembly.

L1: the operating system's ML stack. Windows ML, Core ML, LiteRT. WebNN talks to these.

And the hardware: CPU, GPU, NPU.

The rule of thumb: the higher you go, the less code and the less control. The lower you go, the more control and the more work. We'll walk it top-down, and this little map in the corner tells you where we are.



Slide 8 · L5 · Task APIs: built-in models

Time budget: 1 min (running total 11 min)

Layer 5. Task APIs. You don't pick a model, you don't download anything yourself. You ask the browser to do one job.

Translator and Language Detector use small expert models. Summarizer, Writer, Rewriter and Proofreader use the browser's foundation model, Gemini Nano in Chrome.

They all follow the same shape: check availability, create an instance, use it, destroy it.

One catch to remember: these are desktop features today. Translator and Language Detector don't work on mobile, and the foundation-model APIs run on Windows, macOS, Linux and Chromebook Plus, not on Chrome for Android or iOS.



Slide 9 · Task API status, October 2026

Time budget: 1.5 min (running total 12.5 min)

Here's the honest status, from chromestatus, the Chrome built-in AI docs and Microsoft's Edge docs, as of October 2026.

Translator and Language Detector: stable in Chrome 138 on desktop, and Edge shipped them in version 148 with more than 145 languages.

Summarizer: stable in Chrome 138. In Edge it's a developer preview, on by default in Canary and Dev.

Writer and Rewriter: their origin trials in Chrome ended, so today they are behind a flag again. Edge has them as a developer preview.

Proofreader: same story in Chrome, origin trial ended, flag only. I found no Edge listing for it.

And a bonus row: on-device speech recognition in the Web Speech API, the processLocally option, shipped in Chrome 139.

Firefox and Safari ship none of these. We'll talk about why in a minute.



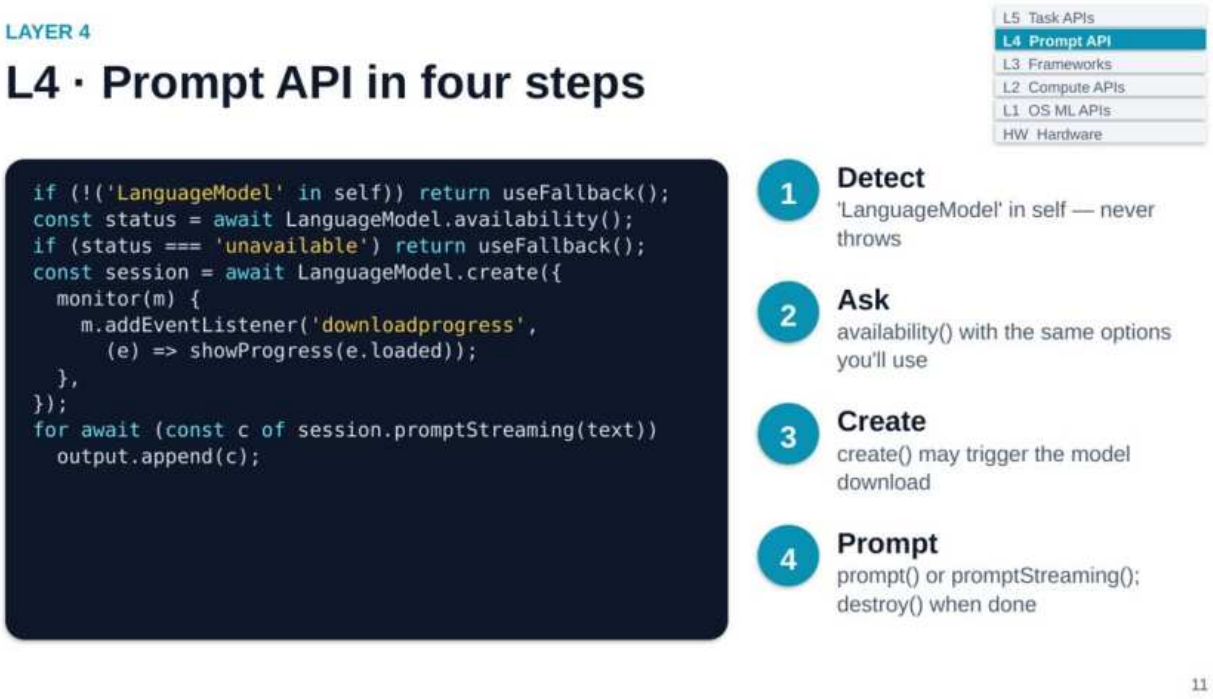
Slide 10 · Translate and summarize

Time budget: 1 min (running total 13.5 min)

And this is all the code. On the left, the Translator: create it with a source and a target language, then call translate.

On the right, the Summarizer: pick a type, key points, tldr, teaser or headline, a format and a length, and pass optional context.

These follow the examples in Chrome's documentation. In real code, call availability() first, because the model might still need to download. That's the pattern I'll show on the next slide.



Slide 11 · L4 · Prompt API in four steps

Time budget: 1.5 min (running total 15 min)

Layer 4, the Prompt API. This is the pattern you will use for every built-in AI API, so let's go slowly.

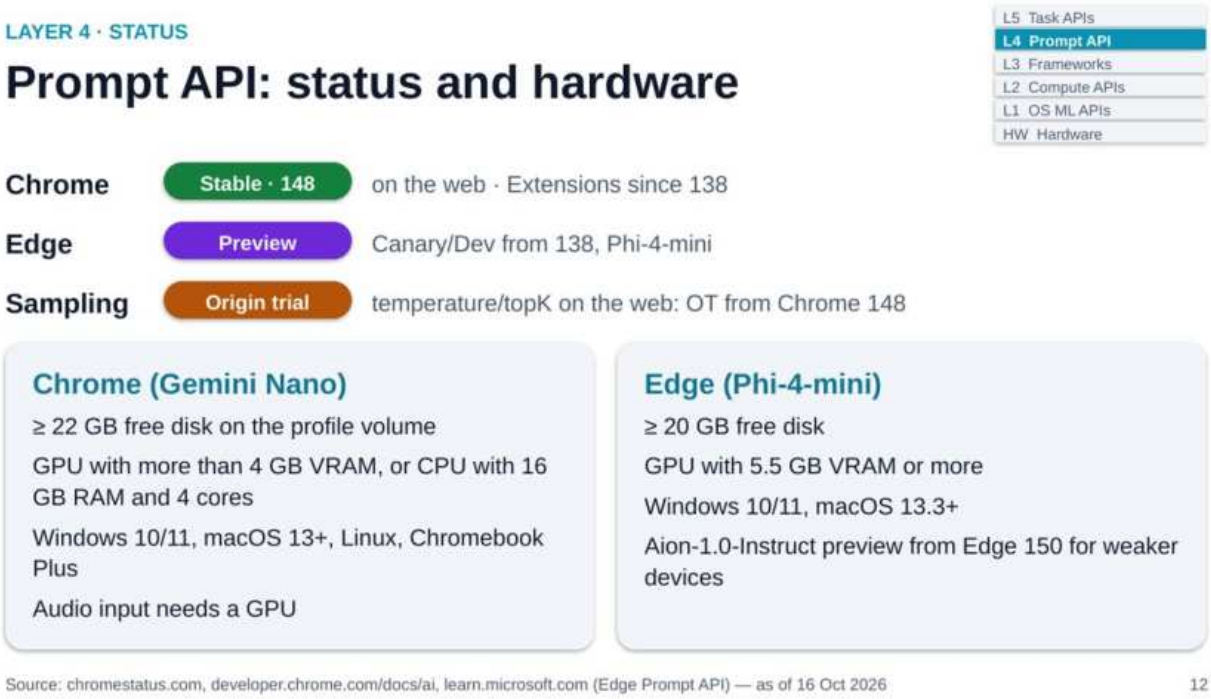
One: detect. 'LanguageModel' in self. That never throws, even in a browser that has never heard of it. If it's missing, go straight to your fallback.

Two: ask. availability() tells you unavailable, downloadable, downloading, or available. Pass the same options you'll use later, for example the languages or input types.

Three: create. This may start the model download, so give the user a progress bar through the monitor callback.

Four: prompt, or promptStreaming to show tokens as they arrive. Destroy the session when you're done, because sessions hold memory and a context window.

This code is almost line for line what runs in the playground's Prompt card.



Slide 12 · Prompt API: status and hardware

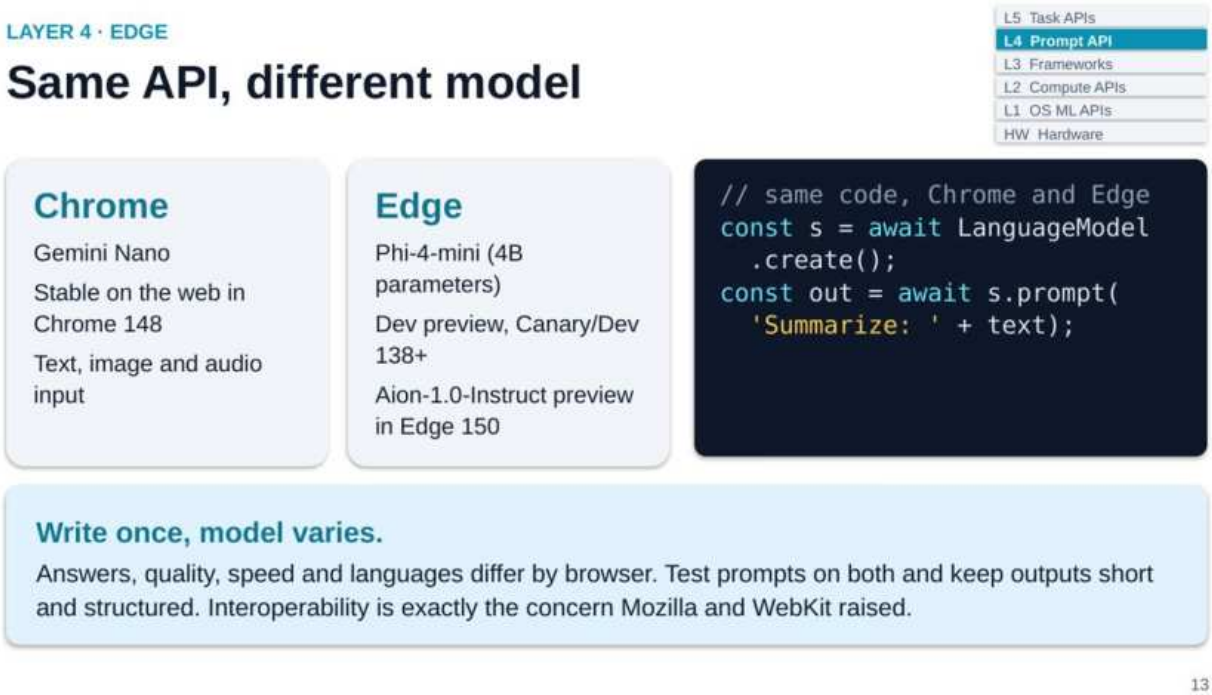
Time budget: 1 min (running total 16 min)

Status: the Prompt API is stable on the web in Chrome 148. Extensions have had it since Chrome 138. In Edge it's a developer preview in Canary and Dev, running Phi-4-mini. Sampling parameters like temperature and topK are in a separate origin trial from Chrome 148. This is from chromestatus and the Chrome and Edge docs, as of October 2026.

Now look at the hardware bar, because this is what decides whether your users actually get it. Chrome wants at least 22 gigabytes of free disk, and either a GPU with more than 4 gigabytes of VRAM or a CPU with 16 gigabytes of RAM and 4 cores. Desktop only. Edge with Phi-4-mini wants 5.5 gigabytes of VRAM.

Edge 150 added a preview model, Aion-1.0-Instruct, that is smaller and runs on weaker GPUs or CPU only.

Translation: plenty of your users will get "unavailable". Plan for it from day one.



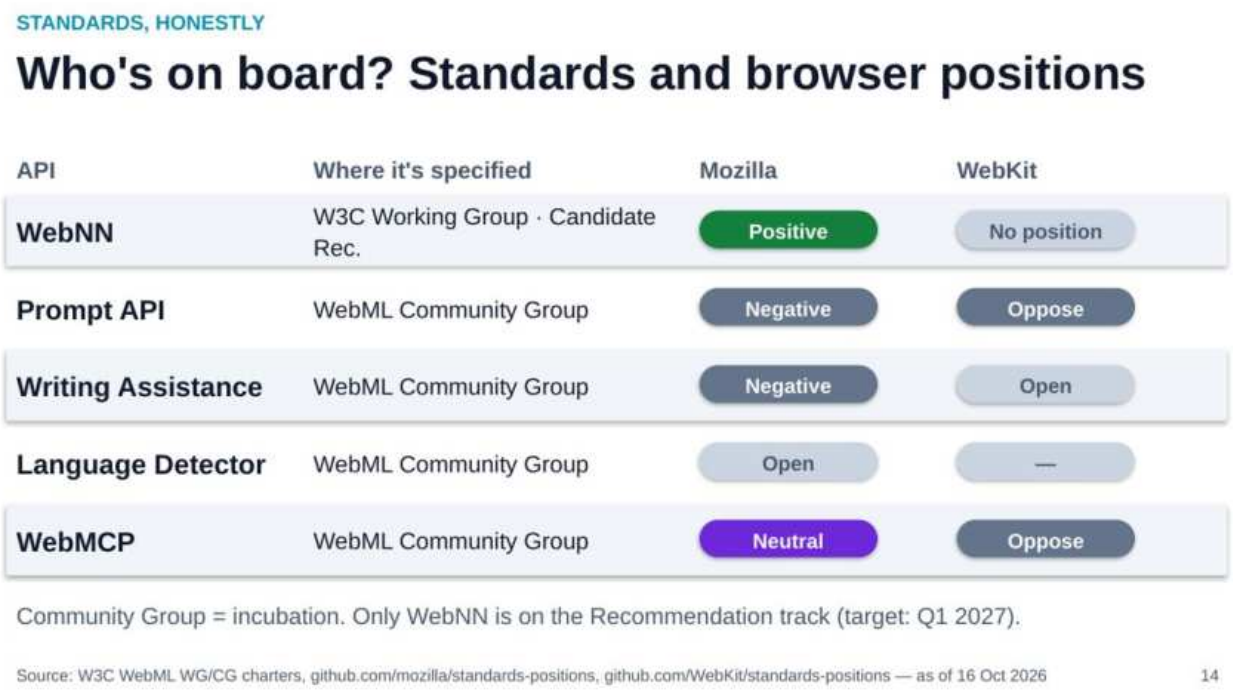
Slide 13 · Same API, different model

Time budget: 1 min (running total 17 min)

Here's something I find fascinating. Edge implements the same Prompt API, same JavaScript, but runs a different model: Phi-4-mini, a 4-billion-parameter model from Microsoft. From Edge 150 there's also a preview model called Aion-1.0-Instruct for weaker hardware.

The code on the right runs unchanged in both browsers. That's great for developers. But the answer you get is not the same. Quality, speed, supported languages, all different.

So: write once, the model varies. Test your prompts in both browsers, keep the output short and structured. And this is exactly the interoperability concern that Mozilla and WebKit raised. Which brings us to standards.



Slide 14 · Who's on board? Standards and browser positions

Time budget: 1 min (running total 18 min)

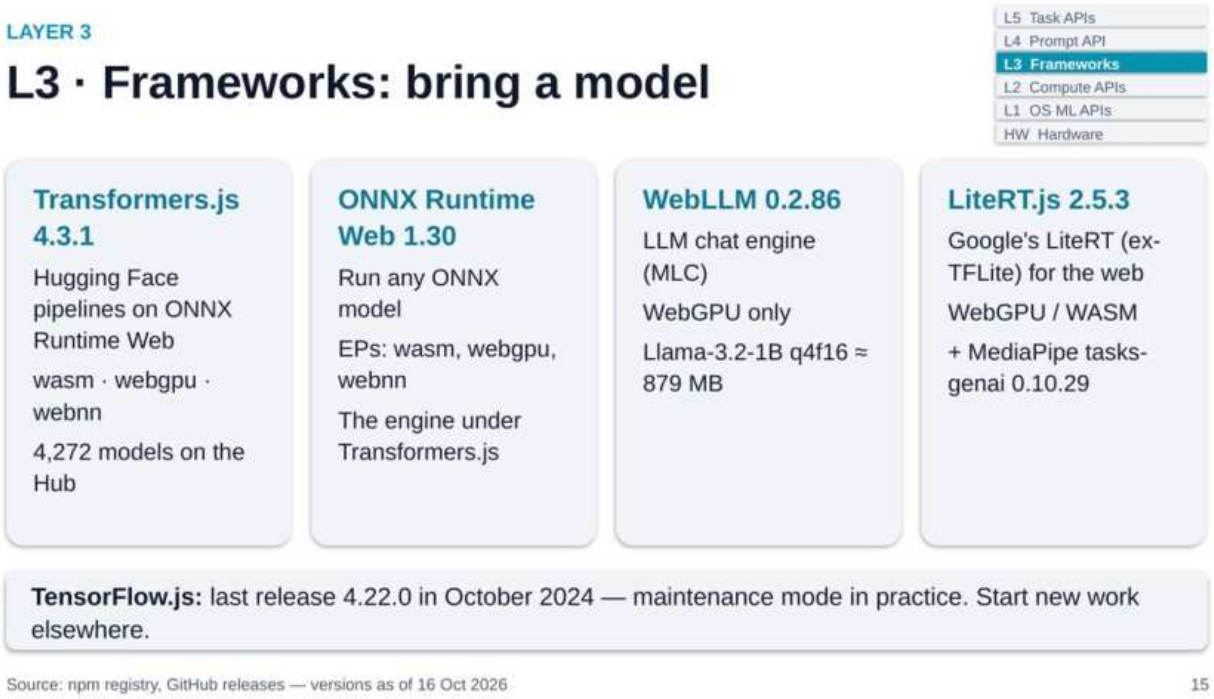
Let me be honest about standards, because this is where marketing and reality differ.

WebNN is a W3C Working Group specification, now a Candidate Recommendation. Mozilla's position is positive. WebKit has no position yet.

Everything else, the Prompt API, the writing assistance APIs, the language detector, WebMCP, lives in the Web Machine Learning Community Group. That's incubation, not the Recommendation track. Yet.

And the other engines are skeptical. Mozilla is negative on the Prompt API and the writing assistance APIs. WebKit opposes the Prompt API, citing interoperability and privacy. On WebMCP, Mozilla is neutral and WebKit opposes.

Sources are the W3C charters and the public standards-positions repos, as of October 2026. My take: use these APIs as progressive enhancement. Never as the only path.



Slide 15 · L3 · Frameworks: bring a model

Time budget: 1 min (running total 19 min)

Layer 3. Frameworks. Here you ship the model yourself, so you choose it, and it works in every browser that has the compute APIs underneath.

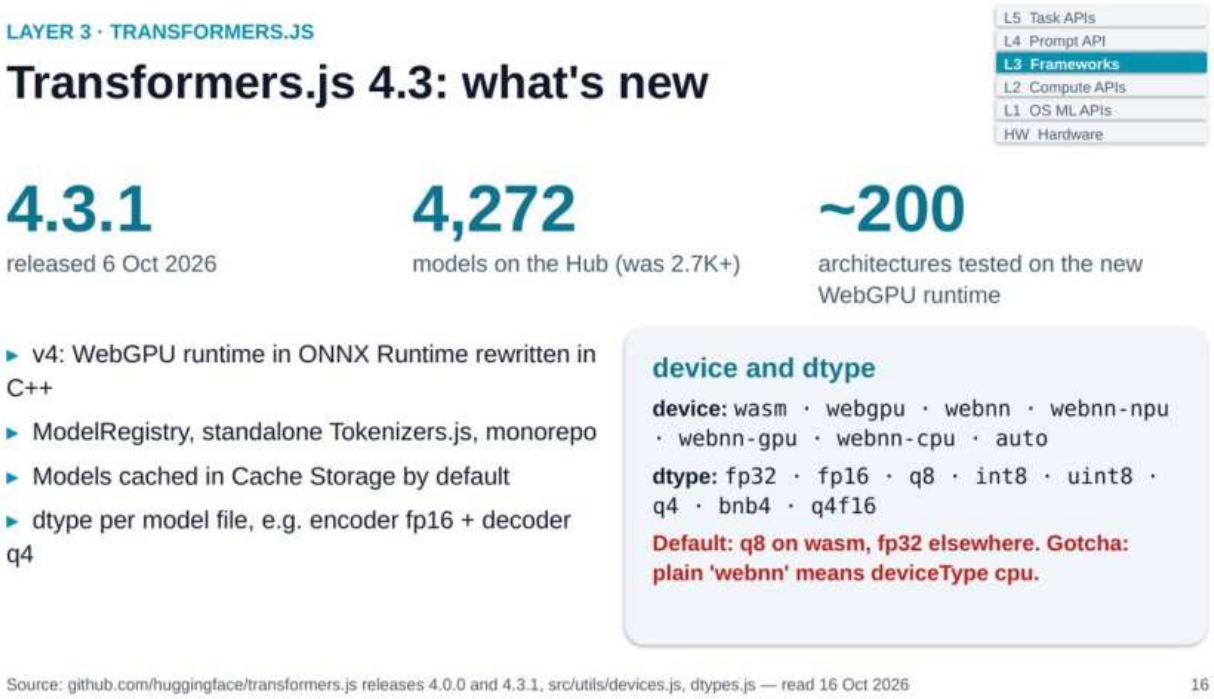
Transformers.js, now 4.3.1: Hugging Face pipelines running on ONNX Runtime Web, with WebAssembly, WebGPU and WebNN backends, and more than 4,200 ready models on the Hub. That's my default, and the demo uses it.

ONNX Runtime Web is the engine underneath. Use it directly if you have your own ONNX model.

WebLLM is a dedicated LLM chat engine. WebGPU only, and the models are big: Llama 3.2 1B is about 880 megabytes.

LiteRT.js is Google's runtime, the former TensorFlow Lite, together with MediaPipe's GenAI tasks.

And TensorFlow.js: its last release was October 2024. I loved it, but for new projects, start elsewhere.



Slide 16 · Transformers.js 4.3: what's new

Time budget: 1.5 min (running total 20.5 min)

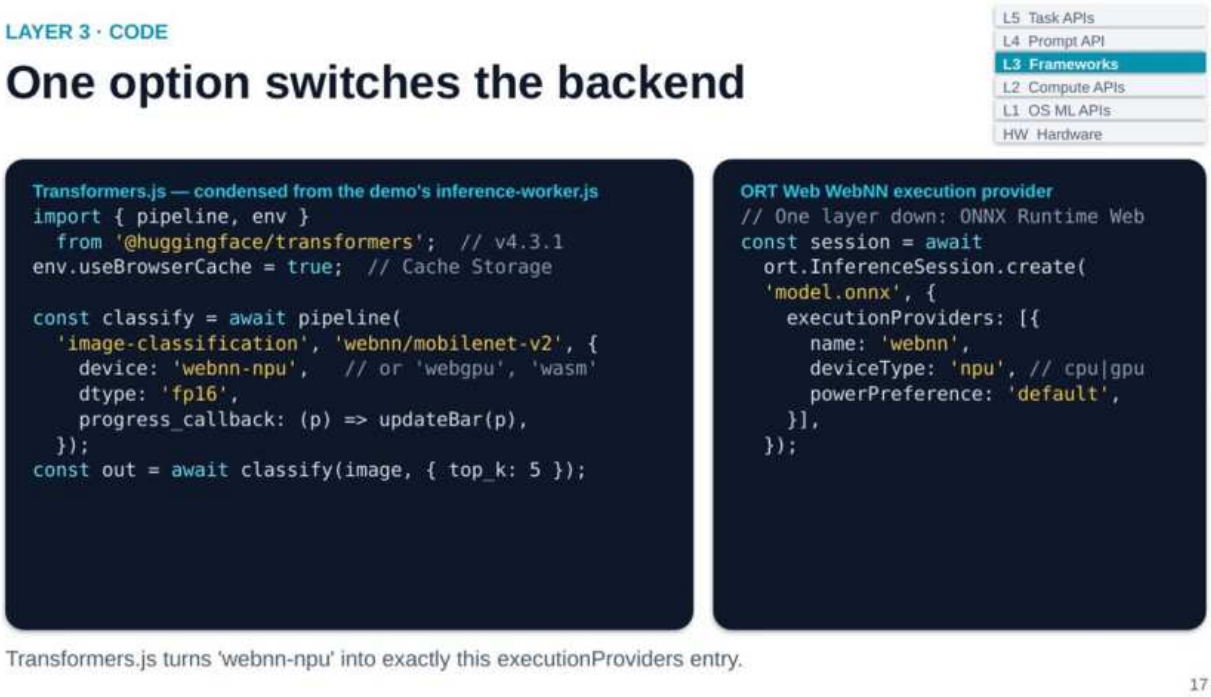
Transformers.js has moved a lot since my old demo, which was on 3.4.2.

Version 4 came out in March 2026; the latest is 4.3.1 from October 6th. The headline change: a new WebGPU runtime inside ONNX Runtime, rewritten in C++, tested across about 200 model architectures. Plus a model registry, a standalone tokenizers library, and a monorepo.

There are now more than 4,200 models on the Hub tagged for Transformers.js. My old slide said 2,700.

Two options matter most. device: wasm, webgpu, and four WebNN variants, plus auto. dtype: from fp32 down to 4-bit. The default is q8 on WebAssembly and fp32 on everything else.

And one gotcha I found reading the source: plain 'webnn' maps to deviceType cpu. If you want the NPU, say webnn-npu explicitly. Also, 'auto' only checks whether the API exists; it doesn't probe the hardware.



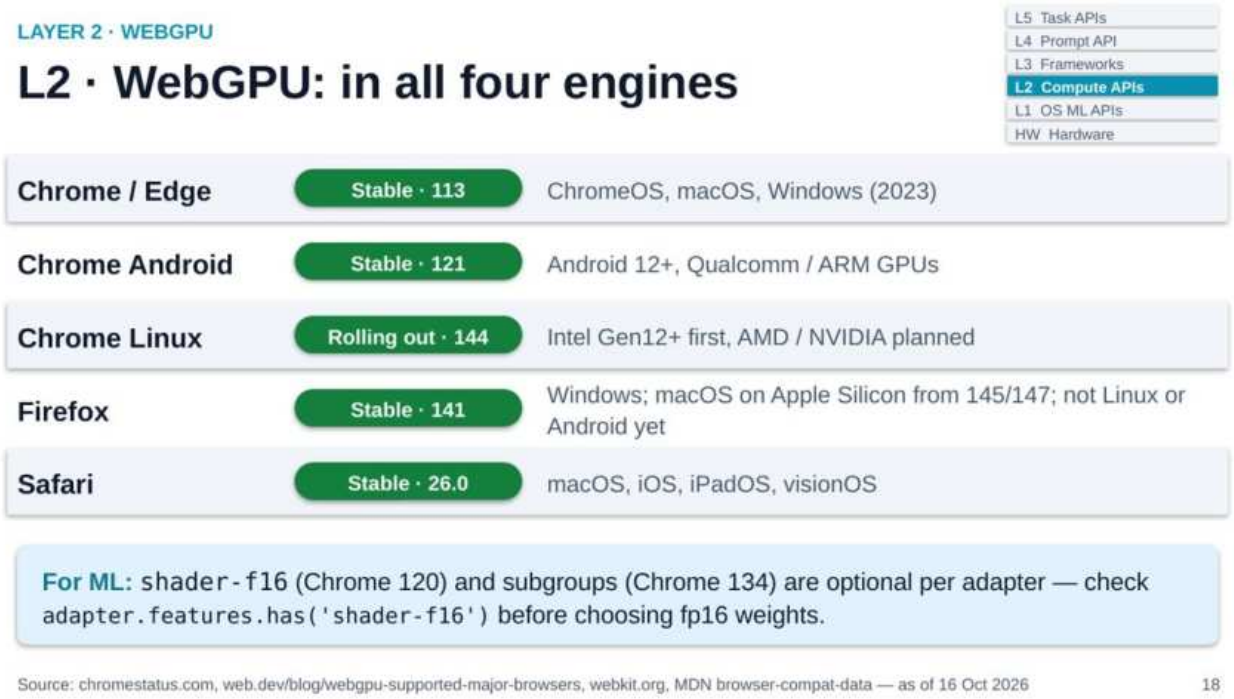
Slide 17 · One option switches the backend

Time budget: 1 min (running total 21.5 min)

Here's the whole thing in code. On the left, condensed from the demo's worker: import Transformers.js 4.3.1, keep the browser cache on, create a pipeline with a task, a model, a device and a dtype, and a progress callback for the download bar. Then call it.

Changing one string, device, moves this from WebAssembly to WebGPU to the NPU. That's the switcher you saw in the demo.

On the right is one layer down: ONNX Runtime Web with the WebNN execution provider. deviceType cpu, gpu or npu, plus a power preference. When you pass webnn-npu to Transformers.js, it builds exactly this entry for you.



Slide 18 · L2 · WebGPU: in all four engines

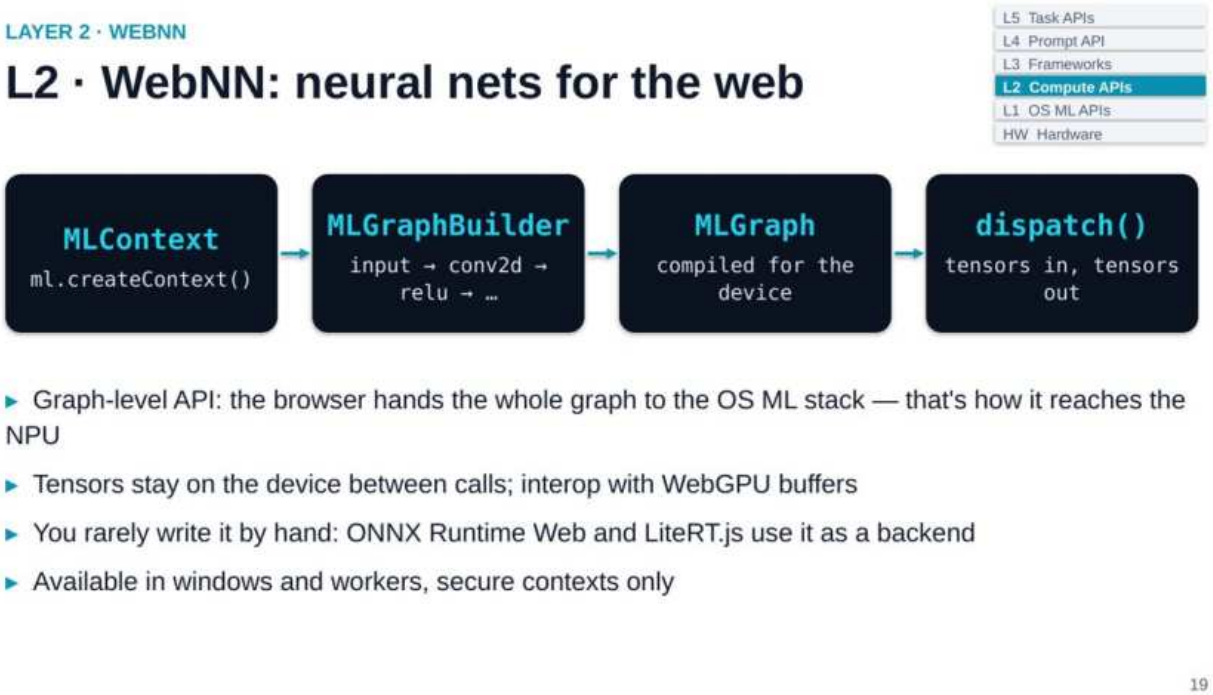
Time budget: 1 min (running total 22.5 min)

Layer 2, starting with WebGPU. This is the good news slide. As of late 2025, WebGPU is supported in Chrome, Edge, Firefox and Safari.

Chrome and Edge since 113 in 2023. Android since 121. Linux is rolling out from Chrome 144, Intel GPUs first. Firefox shipped in 141 on Windows, then macOS on Apple Silicon. Safari 26 shipped it on macOS, iOS, iPadOS and visionOS.

There are still gaps: Firefox on Linux and Android, older macOS for Safari. That's from chromestatus, web.dev, WebKit and MDN, as of October 2026.

For machine learning, two optional features matter: shader-f16 and subgroups. They depend on the adapter, so check before you pick fp16 weights. WebGPU is today's honest default fast path. WebAssembly is the universal fallback.



Slide 19 · L2 · WebNN: neural nets for the web

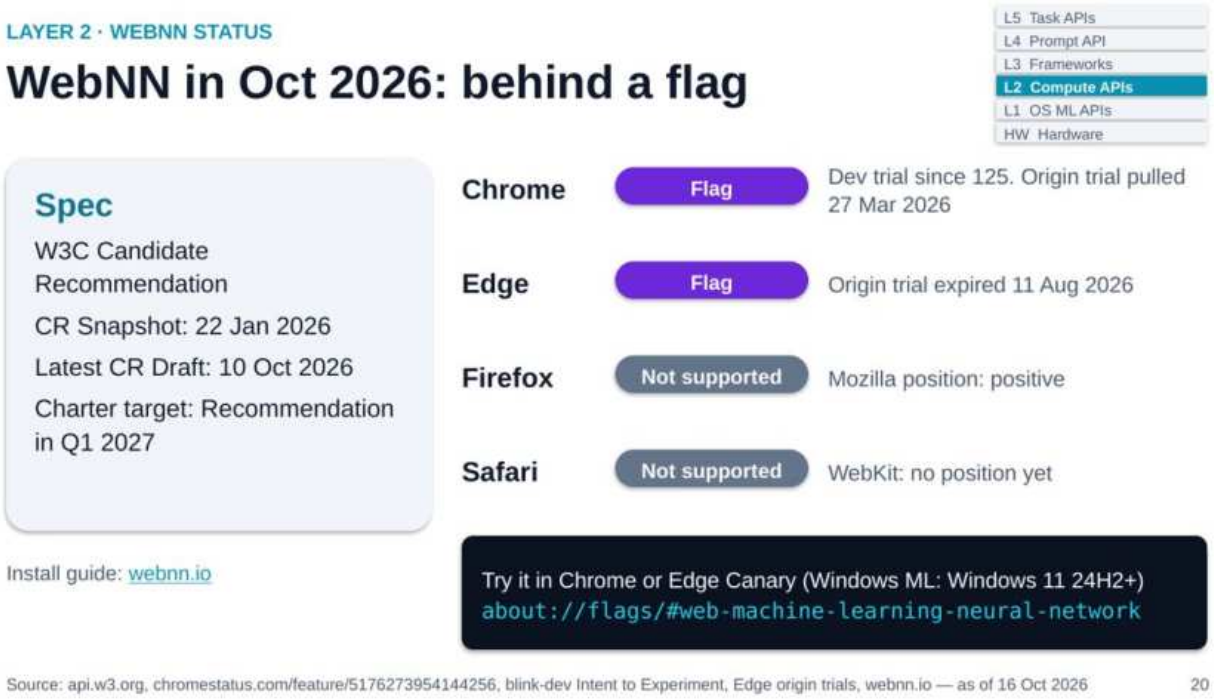
Time budget: 1 min (running total 23.5 min)

Now WebNN. What is it, really?

It's a graph API. You create a context, build a graph of operations, convolution, relu and so on, compile it for a device, and dispatch tensors through it.

The key idea: the browser hands the whole graph to the operating system's ML stack. That's why WebNN can reach the NPU while WebGPU cannot. Tensors can stay on the device between calls, and there's interop with WebGPU buffers.

But honestly, you'll rarely write WebNN by hand. ONNX Runtime Web and LiteRT.js use it as a backend, and Transformers.js uses it through ONNX Runtime. It works in windows and workers, secure contexts only.



Slide 20 · WebNN in Oct 2026: behind a flag

Time budget: 1.5 min (running total 25 min)

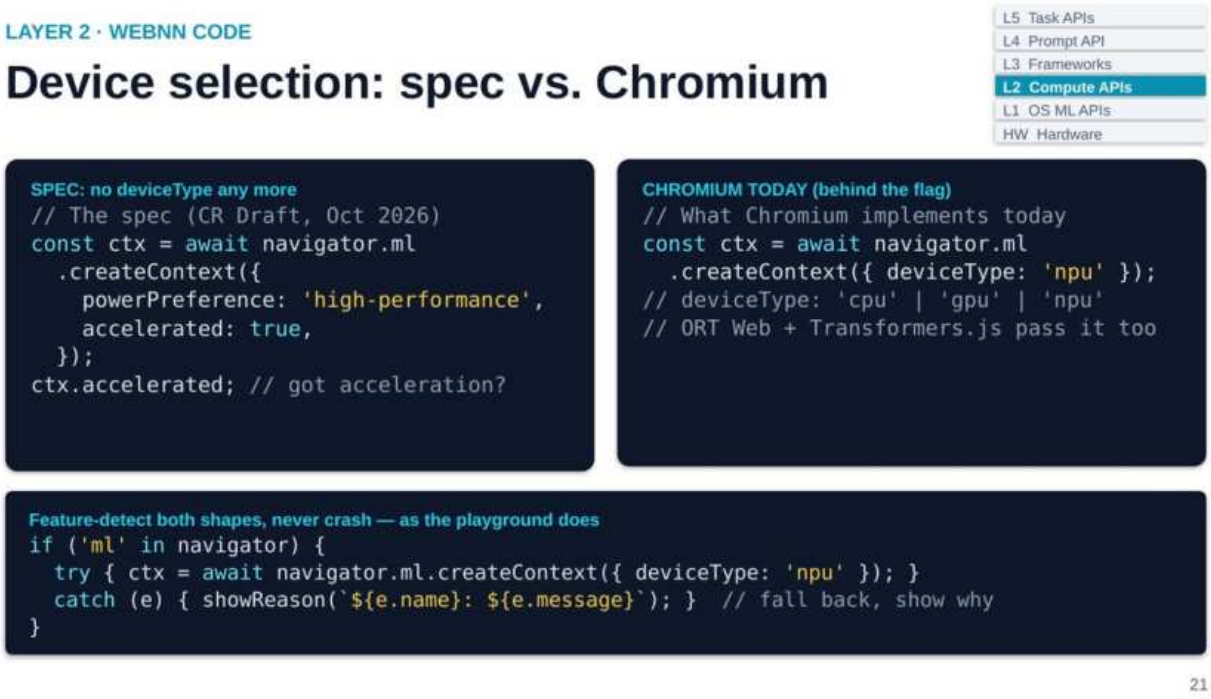
Here's the slide I had to rewrite completely. My old deck said "origin trials until July 2026". That's gone.

The spec is in good shape: W3C Candidate Recommendation, snapshot from January 2026, a fresh draft from October 10th, and the Working Group charter targets a Recommendation in the first quarter of 2027.

The browsers are the bumpy part. Chrome has had a developer trial since version 125. The origin trial was approved, delayed twice, re-enabled, and then on March 27th 2026 it was disabled again because of release-blocking issues. Edge's origin trial expired on August 11th. So today, in both, WebNN is behind a flag. Firefox doesn't implement it but its position is positive. WebKit has no position yet.

This is from chromestatus, the blink-dev intent thread and the Edge origin trials page, as of October 2026.

To try it: Chrome or Edge Canary, turn on this flag. For the NPU path on Windows you need Windows 11 24H2 or newer. webnn.io has the install guide.



Slide 21 · Device selection: spec vs. Chromium

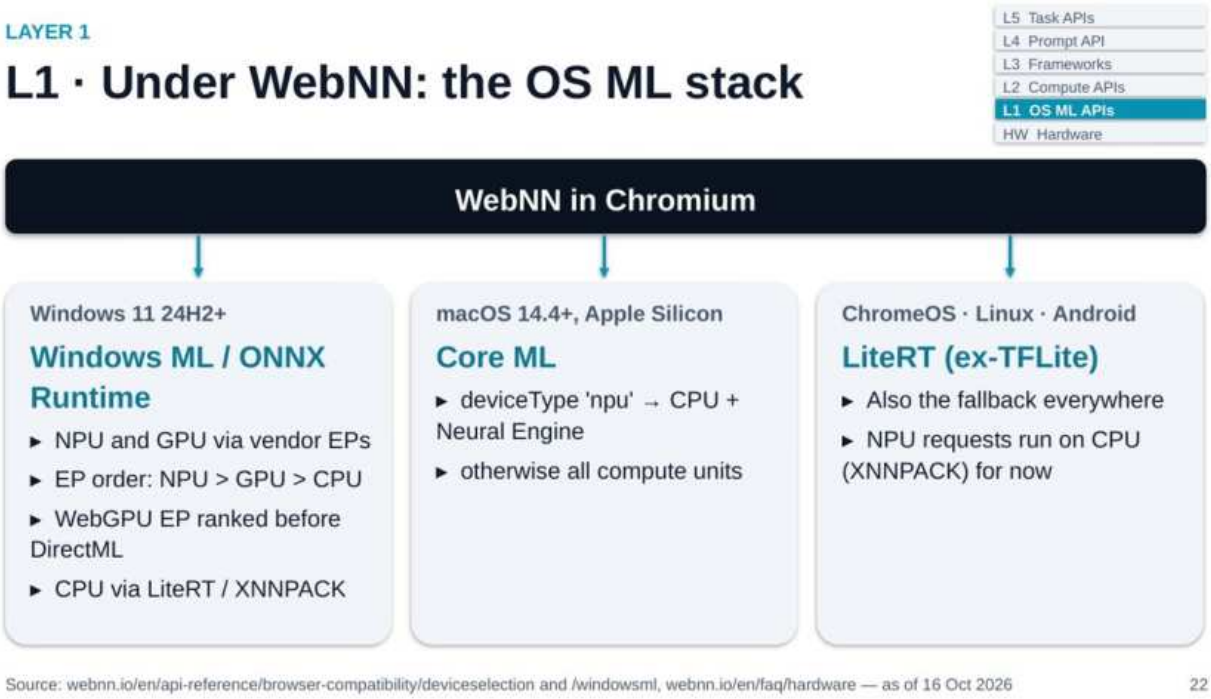
Time budget: 1 min (running total 26 min)

Device selection is where the spec and the implementation disagree right now, so let me show both.

The spec removed deviceType. You now say what you prefer, high-performance or low-power, and whether you want acceleration, and the context tells you if you got it.

But Chromium today, behind the flag, still accepts deviceType cpu, gpu or npu, and does not have the accelerated option yet. ONNX Runtime Web and Transformers.js still pass deviceType.

So in practice: check 'ml' in navigator, try to create the context, and catch the error. The playground probes both shapes and prints each result, so on stage you see exactly what this browser supports. The spec itself says this part is under active development, so expect it to move.



Slide 22 · L1 · Under WebNN: the OS ML stack

Time budget: 1 min (running total 27 min)

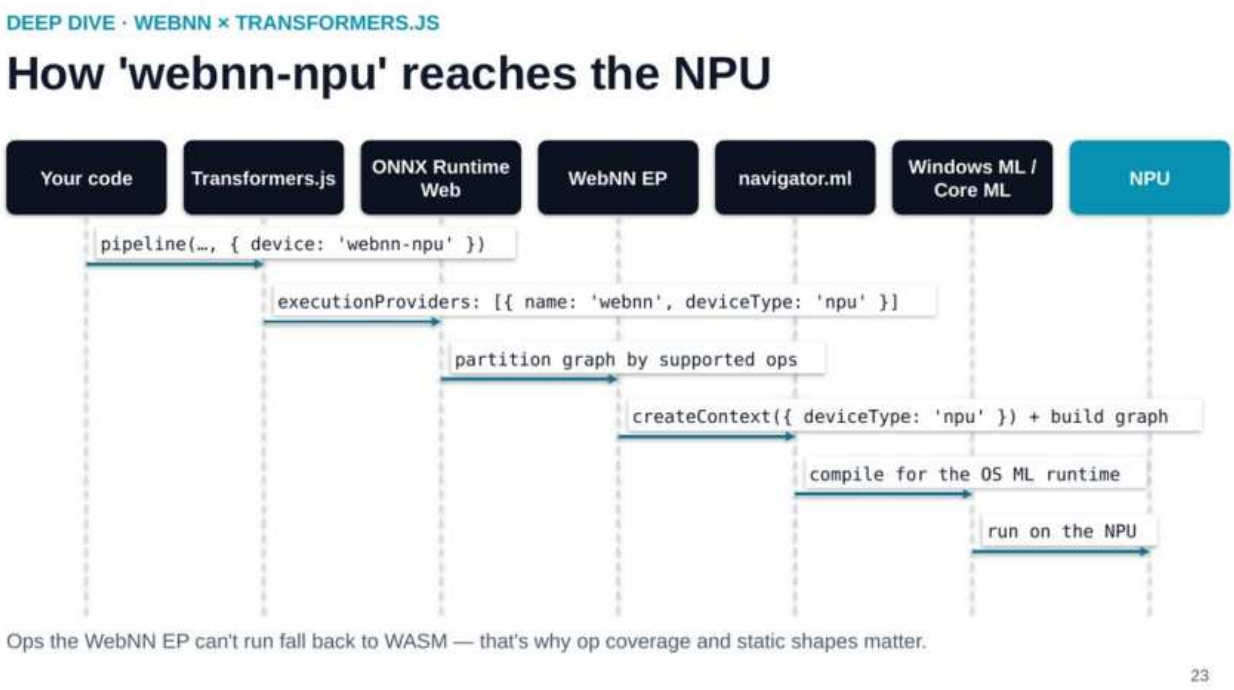
Layer 1. What happens under WebNN depends on your operating system.

On Windows 11 24H2 and newer, Chromium hands the graph to Windows ML, which is ONNX Runtime with vendor execution providers. The order is NPU, then GPU, then CPU. Note that DirectML is now just one option, ranked after the WebGPU execution provider. That's a change from my old slide.

On Apple Silicon Macs with macOS 14.4 or newer, it's Core ML, and asking for the NPU maps to CPU plus Neural Engine.

Everywhere else, and as the fallback, it's LiteRT, the former TensorFlow Lite. And an important footnote: on LiteRT, NPU requests are not accelerated yet; they run on the CPU.

The verified NPU list on webnn.io is Intel Core Ultra, from Meteor Lake to Panther Lake. Sources are the webnn.io compatibility pages as of October.

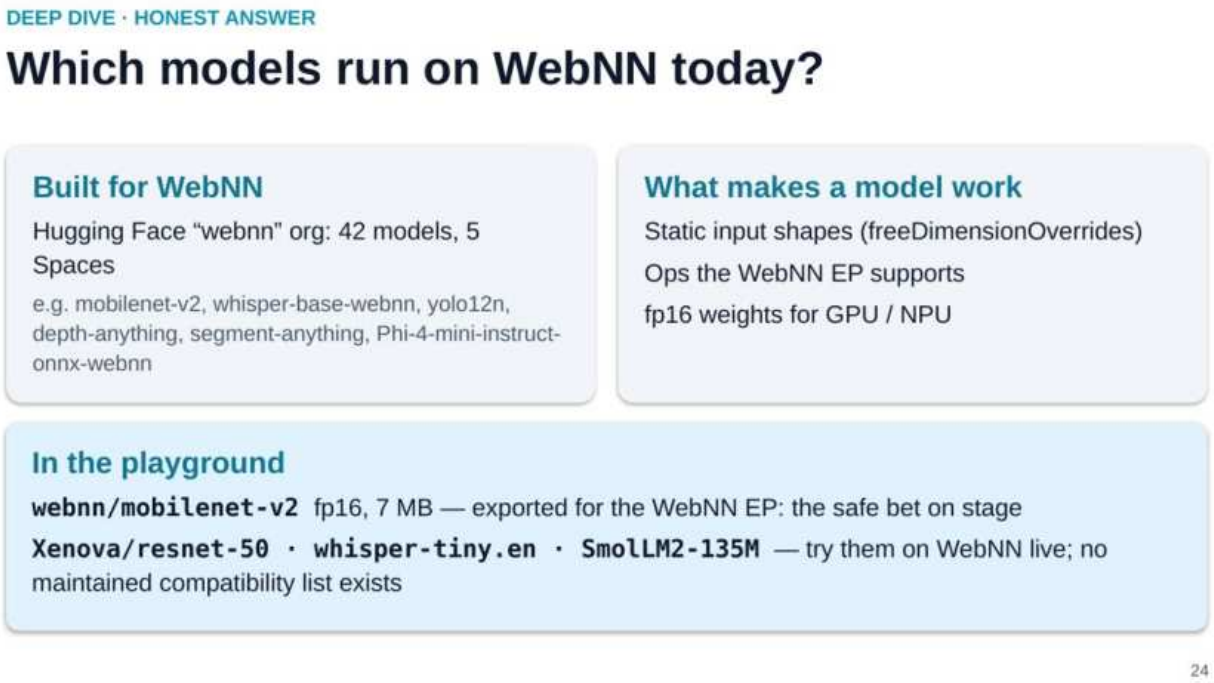


Slide 23 · How 'webnn-npu' reaches the NPU

Time budget: 1 min (running total 28 min)

This is the path a single string travels. Your code says device webnn-npu. Transformers.js turns that into an ONNX Runtime execution provider entry: name webnn, deviceType npu. ONNX Runtime partitions the model: the operations the WebNN provider supports go to WebNN, the rest stays on WebAssembly. The WebNN provider creates a context with deviceType npu and builds the graph. Chromium compiles it for Windows ML or Core ML. And it runs on the NPU.

Two things follow from this picture. First, op coverage matters: anything the provider can't run falls back to the CPU, and those hops cost time. Second, the WebNN provider wants static input shapes, which is why there are special exports for WebNN. Next slide.



Slide 24 · Which models run on WebNN today?

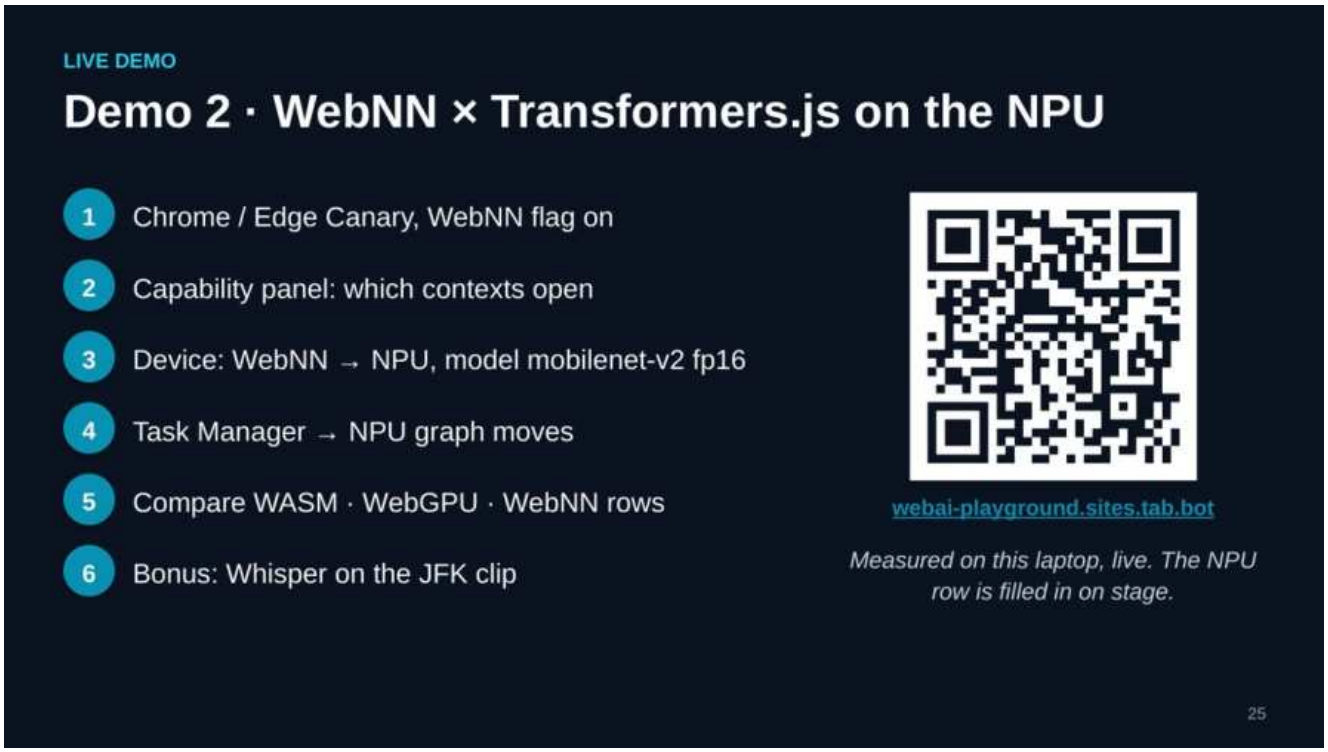
Time budget: 1 min (running total 29 min)

So which models actually run on WebNN today? Honest answer: there is no maintained compatibility list. I looked.

What exists is the Hugging Face "webnn" organization: 42 models and 5 Spaces exported specifically for WebNN. MobileNet, Whisper base, YOLO, Depth Anything, Segment Anything, even Phi-4-mini.

What makes a model work: static input shapes, operations the WebNN provider supports, and usually fp16 weights for GPU and NPU.

In the playground I use webnn/mobilenet-v2, fp16, 7 megabytes, built for the WebNN provider. That's my safe bet on stage. ResNet-50, Whisper and SmolLM2 you can try on WebNN live; whether they run fully on the NPU depends on your browser build and drivers. Let's find out.



Slide 25 · Demo 2 · WebNN × Transformers.js on the NPU

Time budget: 3.5 min (running total 32.5 min)

[Switch to Chrome Canary or Edge Canary with about://flags/#web-machine-learning-neural-network enabled. Windows 11 24H2+. Open Task Manager → Performance → NPU, and keep it visible next to the browser.]
[Unverified until you run it: WebNN was not available on our Linux test box, so the NPU path has never been tested end to end. Rehearse it on this exact laptop and browser build.]

Now the fun part. Same playground, but in Canary with the WebNN flag on.

[Point at the capability panel.] Look: navigator.ml exists, and the panel shows which createContext calls succeeded. deviceType npu: yes or no, right there.

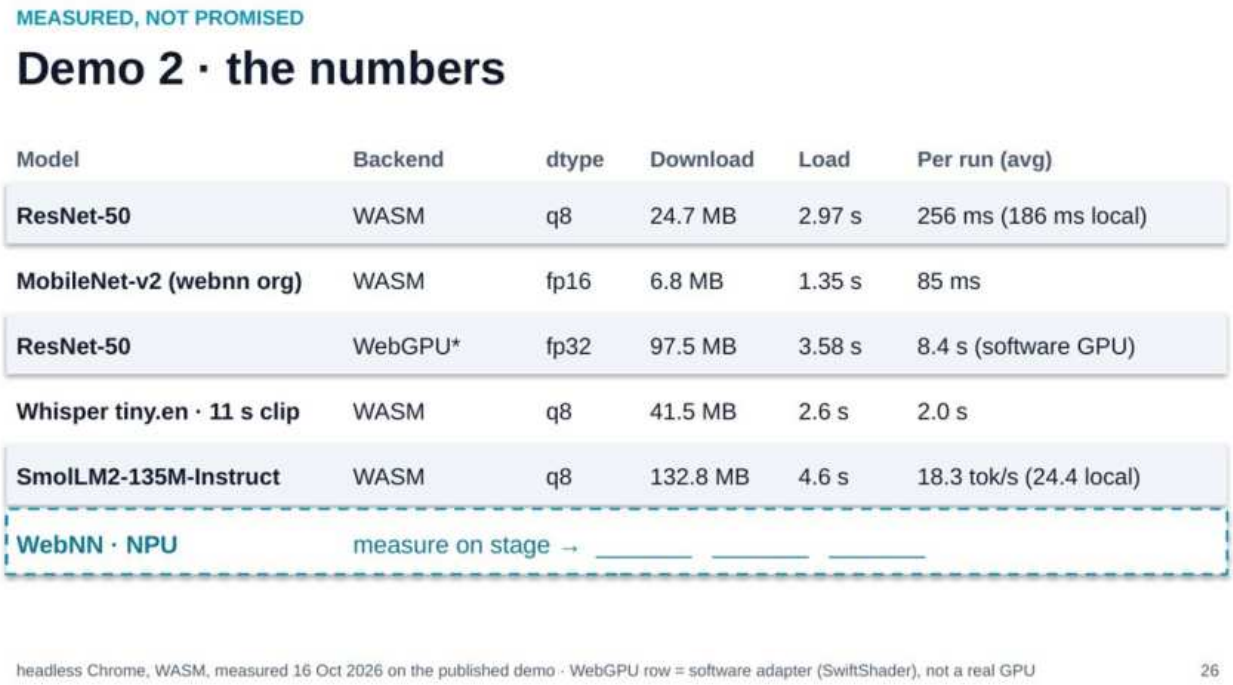
[Device: WebNN, sub-picker NPU. Model: mobilenet-v2 fp16. Classify the coffee cup a few times.]

And watch Task Manager. *[Point at the NPU graph.]* That's the NPU working. Not the GPU, not the CPU.

[Scroll to the benchmark table.] Now we have three rows: WASM, WebGPU and WebNN on the NPU, same page, same model family. Read out the averages.

[If time allows: speech card, Whisper on the JFK clip, on WebGPU or WebNN.]

If the NPU path fails on a model, the panel shows the error text. That's the honest state of WebNN in October 2026, and it's also why we always keep a fallback.



Slide 26 · Demo 2 · the numbers

Time budget: 1 min (running total 33.5 min)

For the record, here are numbers we measured before the conference, on the published demo in headless Chrome. Not my laptop: a cloud machine with no GPU.

ResNet-50, 8-bit, on WebAssembly: about a quarter of a second per image, 186 milliseconds on a local run. MobileNet-v2 from the webnn org: 85 milliseconds. Whisper tiny transcribes an 11-second clip in about 2 seconds. SmolLM2 135M generates 18 to 24 tokens per second.

The WebGPU row looks terrible, 8 seconds, because that machine had only a software GPU. On a real GPU you'd expect the opposite. I keep it on the slide on purpose: always check what adapter you actually got.

And the last row is empty on purpose. That's the number we just measured live on the NPU. *[Read the WebNN NPU average from the demo's benchmark table and say it out loud.]*



Slide 27 · Prompt API: images, audio, JSON

Time budget: 1 min (running total 34.5 min)

The Prompt API that shipped in Chrome 148 is not text-only. It takes images and audio as input. Output is still text.

And the feature I like most for real apps: response constraints. Pass a JSON Schema, or a regular expression, and the model's answer has to match it. No more parsing free-form text and praying.

Declare the input types you'll use when you create the session, and pass the same options to availability(), so you get an honest answer. And remember: audio input needs a GPU.

The message format, a role plus a content array of typed parts, is the one in Chrome's Prompt API documentation.



Slide 28 · Demo 3 · Prompt API with a fallback

Time budget: 2.5 min (running total 37 min)

[Switch to Chrome stable (148+) on a machine that meets the hardware bar. Open the Prompt card.]

Last demo. The Prompt card. First, the capability panel: LanguageModel exists, and availability() says... *[read it out: available, downloadable or downloading]*.

[Type a short prompt, e.g. "Explain WebNN in one sentence", click Run.]

If the model still needs to download, you see the progress bar from the monitor callback. Then tokens stream in from Gemini Nano, on this device.

[Now switch to a browser without the API, or one where availability() says unavailable, such as Firefox or a profile with the flag off. Run the same prompt.]

Same card, same button. Now availability is unavailable, so the page falls back to Transformers.js with SmolLM2, a 135-million-parameter model, running in a Web Worker. It's not as smart as Gemini Nano, but the feature still works. Same UI, different engine.

[On our test machine LanguageModel existed but availability() returned "unavailable", and the fallback ran at about 18 tokens per second. That's the screenshot on the slide.]

That's the pattern I want you to steal: detect, prefer the built-in model, fall back to your own.



Slide 29 · Which layer for which job?

Time budget: 1 min (running total 38 min)

So how do you choose? Here's my decision matrix.

Translation and language detection: use the built-in task APIs where they exist. Fall back to a translation model in Transformers.js.

Summarize, rewrite, proofread: built-in task APIs first, then the Prompt API, then a small model of your own.

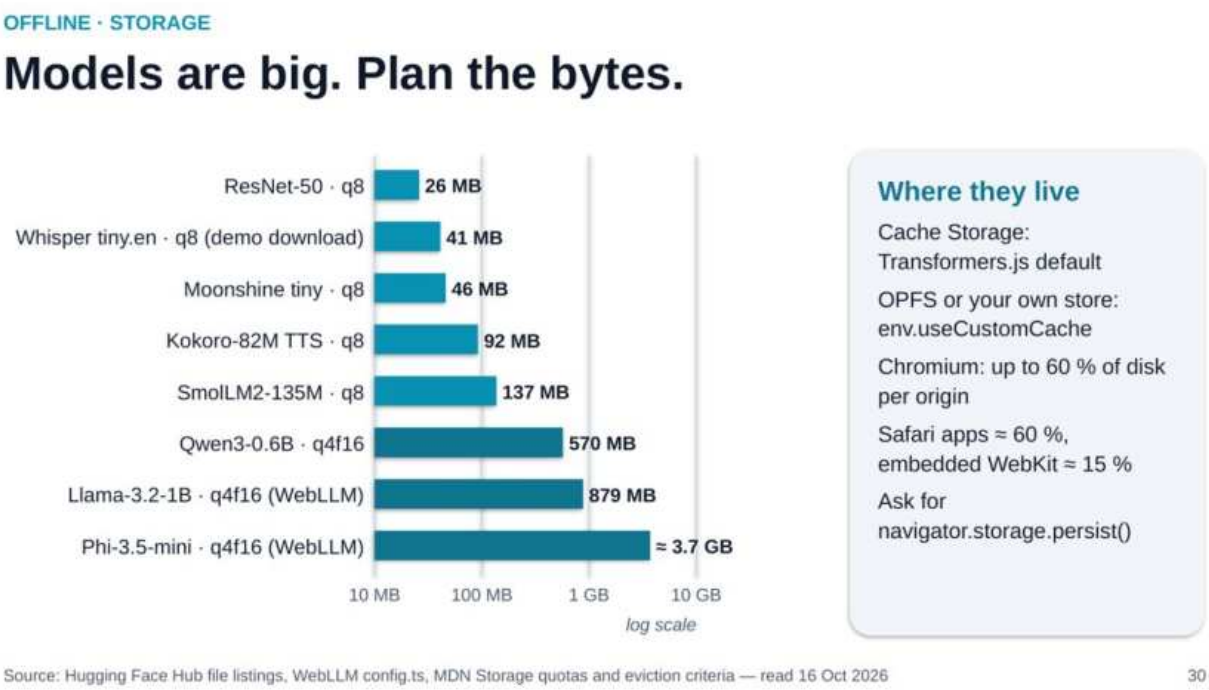
Chat, extraction, structured JSON: the Prompt API, falling back to Transformers.js or WebLLM, or to the cloud if the device can't handle it.

Vision: there's no built-in API, so Transformers.js on WebNN or WebGPU, with the same model on WebAssembly as the fallback.

Speech-to-text: Web Speech with processLocally where it's available, Whisper or Moonshine in Transformers.js otherwise.

Your own model: ONNX Runtime Web directly.

The rule: take the highest layer that does the job, and always have the next one down ready.



Slide 30 · Models are big. Plan the bytes.

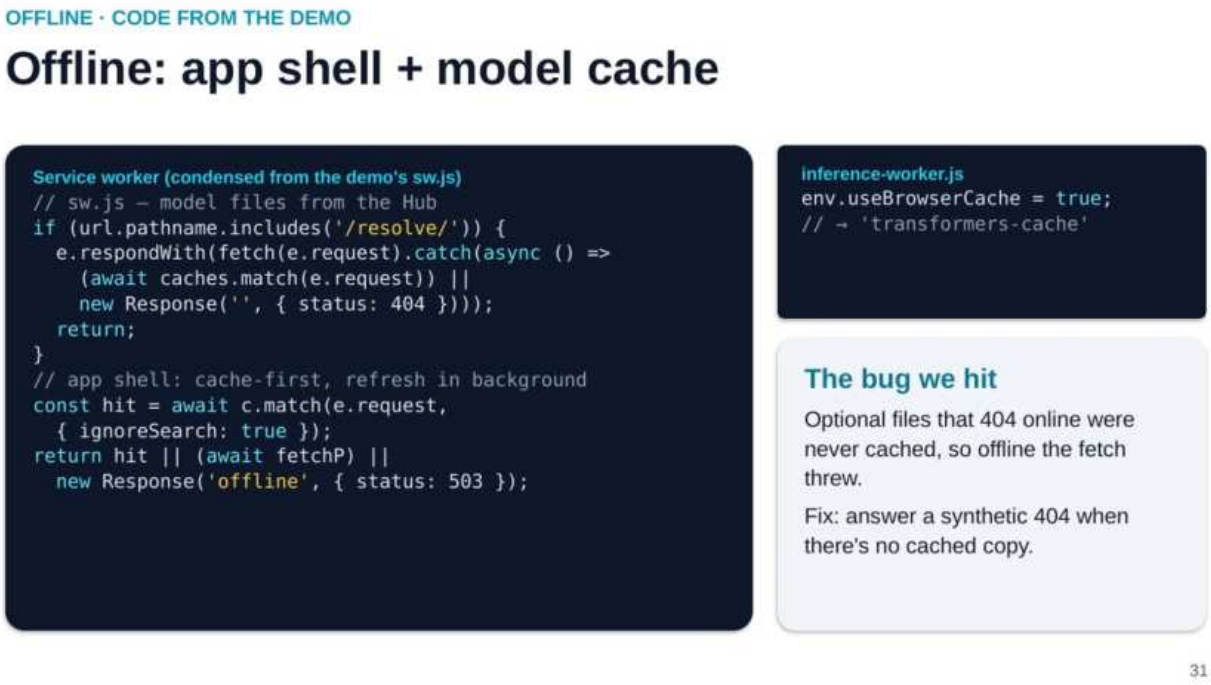
Time budget: 1 min (running total 39 min)

Offline starts with bytes. Here are real model sizes from the Hugging Face Hub, on a log scale.

ResNet-50 at 8 bits: 26 megabytes. Whisper tiny, what the demo downloads: about 41. Kokoro text-to-speech: 92. SmolLM2 135M: 137. Then the language models jump: Qwen3 0.6B at 4 bits is 570 megabytes, Llama 3.2 1B in WebLLM almost 900, Phi-3.5 mini about 3.7 gigabytes.

Where do they live? Transformers.js uses Cache Storage by default; you can plug in your own store, such as OPFS. Quotas are generous on desktop: Chromium lets an origin use up to 60 percent of the disk, and so do Safari apps, while embedded WebKit gets about 15 percent. Ask for persistent storage so the browser doesn't evict your models. That's from MDN's storage quota page.

My rule: if the model is bigger than the page's total value to the user, don't auto-download it. Ask.



Slide 31 · Offline: app shell + model cache

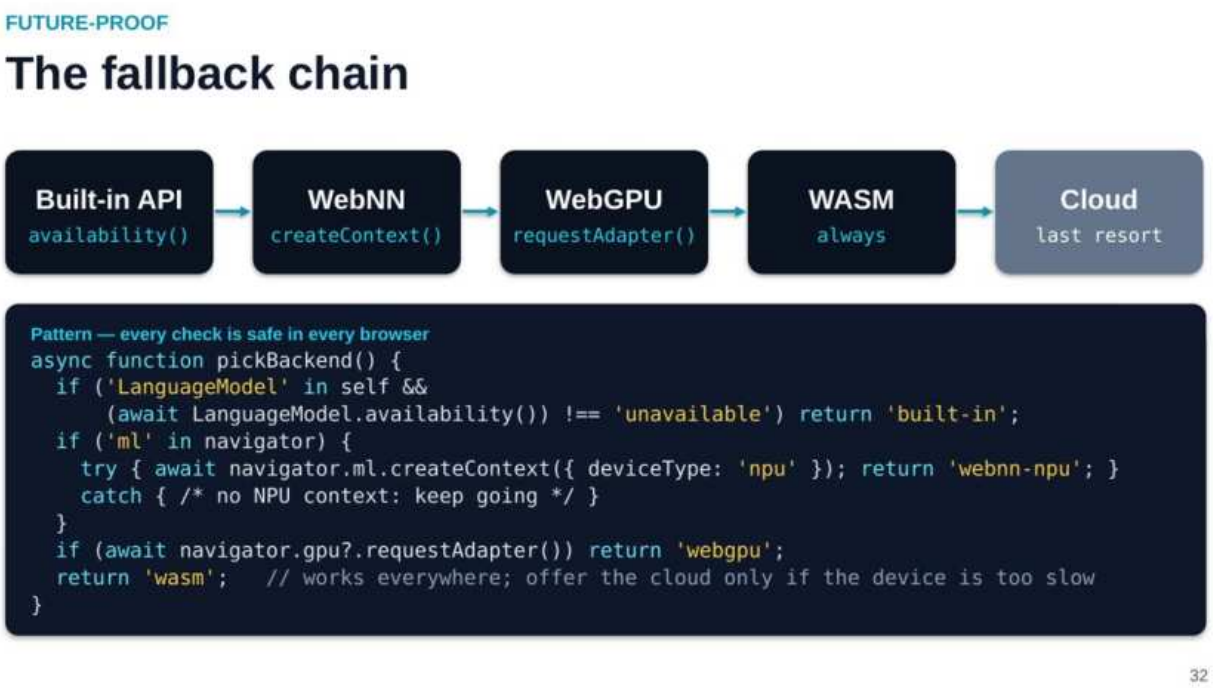
Time budget: 1 min (running total 40 min)

Here's the offline part, straight from the demo.

The service worker precaches the app shell and serves it cache-first. Model files from the Hugging Face Hub are cached by Transformers.js itself, in a cache called transformers-cache; the service worker just makes sure that, when the network is gone, the request gets the cached copy.

And here's a real bug we hit while building this demo, worth sharing. Transformers.js asks for some optional files that simply don't exist on the Hub; online they return 404. A 404 never gets cached. So offline, that fetch threw an error and the whole model load aborted. The fix is three lines: when there's no cached copy, answer with a synthetic 404 instead of failing. After that, ResNet-50 loaded from cache in under a second and classified with the network off.

Lesson: test offline for real, with the network actually off.



Slide 32 · The fallback chain

Time budget: 1 min (running total 41 min)

Put it together and you get a fallback chain. Built-in API if availability says yes. Otherwise WebNN if a context opens on the NPU. Otherwise WebGPU if there's a real adapter. Otherwise WebAssembly, which works everywhere. And only then, if the device is simply too weak, the cloud.

Every check here is safe in every browser: none of them throws in a browser that doesn't know the API. That's progressive enhancement, applied to AI.

And this is what makes the app future-proof. When WebNN leaves the flag, or Firefox ships something new, your app picks it up without a redeploy. The playground's Auto mode does exactly this and shows you the reason it picked.



Slide 33 · What's next: WebMCP — tools, not clicks

Time budget: 1 min (running total 42 min)

One slide on what's next. Everything so far was about running models in the page. WebMCP flips it: your page offers tools to the browser's own AI agent.

You register a tool, a JavaScript function with a description and a JSON Schema for its input, and the agent can call it inside the user's session. Add to cart, book a slot, search the catalogue. No screen scraping, no fake clicks.

Status: it's a W3C Community Group draft, updated today, October 16th, and it's in a Chrome origin trial from 149 to 162. Mozilla is neutral, WebKit opposes. Sources on the slide.

I care about this a lot, so I wrote a book about it: "Tools, Not Clicks: Building Agent-Ready Websites with WebMCP". The free online edition is behind that QR code.



Slide 34 · Three things to take home

Time budget: 1 min (running total 43 min)

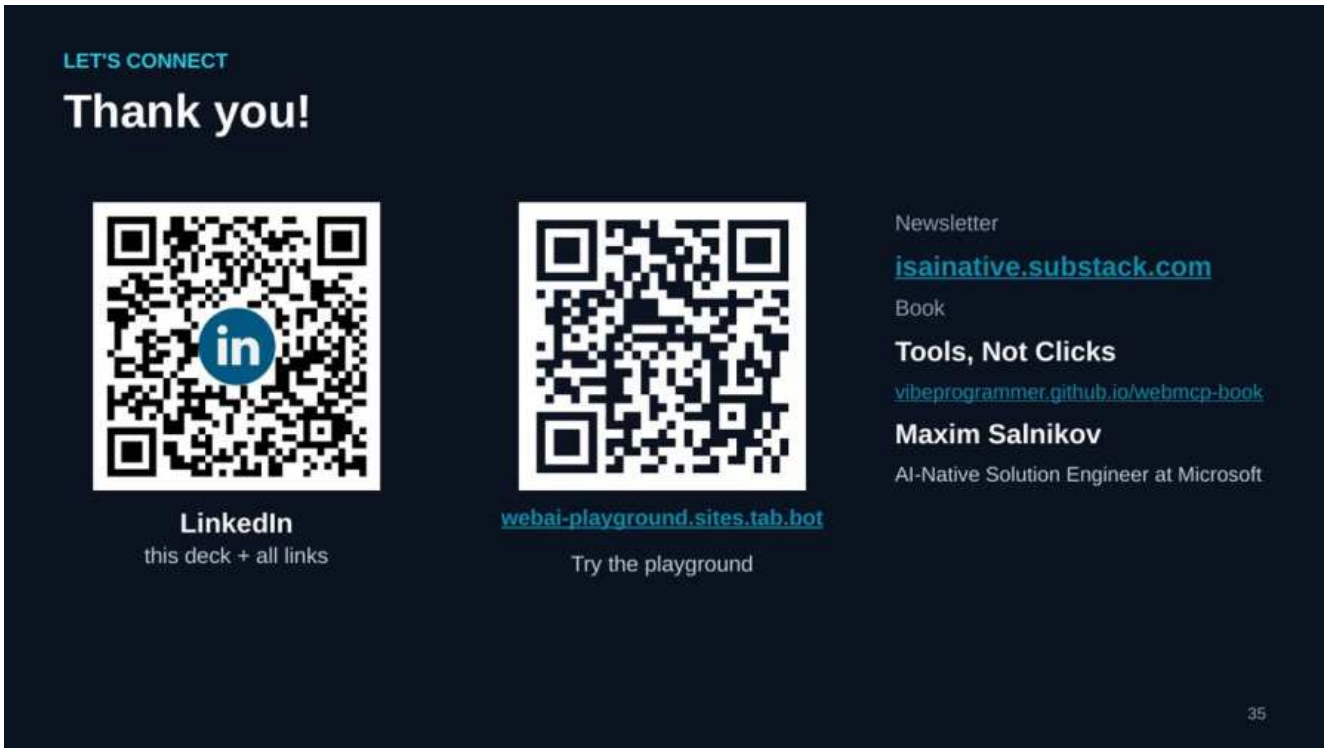
Three things to take home.

One: pick the highest layer that does the job. If a built-in task API does it, use it. Less code, nothing to download.

Two: detect, never assume. availability(), createContext(), requestAdapter(). And WebAssembly always works.

Three: ship it offline. A service worker plus Cache Storage. Treat models as assets, and test with the network really off.

And on WebNN: it's behind a flag today, the spec is a Candidate Recommendation, and a Recommendation is targeted for early 2027. Build the code path now, so your app lights up the NPU the day it ships.



Slide 35 · Thank you!

Time budget: 0.5 min (running total 43.5 min)

Thank you!

Connect with me on LinkedIn, that's the QR on the left, and I'll send you this deck with all the links. The playground is the QR in the middle, it's live, try it on your own laptop tonight, ideally with the WebNN flag on. And if you want more of this, my newsletter is isainative.substack.com.

[Leave this slide up for Q&A.]